

开源软件基础

Beautiful Soup

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 1, 2017



Outline

- 1 快速开始
- 2 对象的种类
- 3 遍历文档树
- 4 搜索文档树

快速开始

下面的一段 HTML 代码将作为例子被多次用到. 这是爱丽丝梦游仙境的一段内容 (以后内容中简称为爱丽丝的文档):

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and the
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

快速开始

使用 BeautifulSoup 解析这段代码, 能够得到一个 BeautifulSoup 的对象, 并能按照标准的缩进格式的结构输出:

```
>>> from bs4 import BeautifulSoup
>>> soup = BeautifulSoup(html_doc)
>>> print(soup.prettify())
<html>
<head>
<title>
  The Dormouse's story
</title>
</head>
<body>
<p class="title">
  <b>
    The Dormouse's story
  </b>
</p>
<p class="story">
  Once upon a time there were three little sisters; and their names were
  <a class="sister" href="http://example.com/elsie" id="link1">
    Elsie
  </a>
  ,
  <a class="sister" href="http://example.com/lacie" id="link2">
    Lacie
  </a>
  and
  <a class="sister" href="http://example.com/tillie" id="link2">
    Tillie
  </a>
```

快速开始

几个简单的浏览结构化数据的方法:

```
>>> soup.title
<title>The Dormouse's story</title>
>>> soup.title.name
u'title'
>>> soup.title.string
u'The Dormouse's story'
>>> soup.title.parent.name
u'head'
>>> soup.p
<p class="title"><b>The Dormouse's story</b></p>
>>> soup.p['class']
u'title'
>>> soup.a
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>
>>> soup.find_all('a')
[<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
>>> soup.find(id="link3")
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

快速开始

从文档中找到所有 `<a>` 标签的链接:

```
for link in soup.find_all('a'):
    print(link.get('href'))
# http://example.com/elsie
# http://example.com/lacie
# http://example.com/tillie
```

快速开始

从文档中获取所有文字内容:

```
>>> print(soup.get_text())
```

```
The Dormouse's story
```

```
The Dormouse's story
```

```
Once upon a time there were three little sisters; and
```

```
Elsie,
```

```
Lacie and
```

```
Tillie;
```

```
and they lived at the bottom of a well.
```

```
...
```

Outline

- 1 快速开始
- 2 对象的种类
- 3 遍历文档树
- 4 搜索文档树

对象的种类

Beautiful Soup 将复杂 HTML 文档转换成一个复杂的树形结构, 每个节点都是 Python 对象, 所有对象可以归纳为 4 种: Tag , NavigableString , BeautifulSoup , Comment .

```
soup = BeautifulSoup('<b class="boldest">Extremely bol  
tag = soup.b  
type(tag)  
# <class 'bs4.element.Tag'>
```

对象的种类

Tag 有很多方法和属性, 在遍历文档树和搜索文档树中有详细解释. 现在介绍一下 tag 中最重要的属性: name 和 attributes

对象的种类

每个 tag 都有自己的名字, 通过.name 来获取:

```
tag.name
```

```
# u'b'
```

对象的种类

一个 tag 可能有很多个属性. tag `<b class="boldest">` 有一个 “class” 的属性, 值为 “boldest”. tag 的属性的操作方法与字典相同:

```
tag['class']  
# u'boldest'
```

对象的种类

也可以直接”点”取属性, 比如: `.attrs` :

```
tag.attrs  
# {u'class': u'boldest'}
```

对象的种类

字符串常被包含在 tag 内.Beautiful Soup 用 NavigableString 类来包装 tag 中的字符串:

```
tag.string  
# u'Extremely bold'  
type(tag.string)  
# <class 'bs4.element.NavigableString'>
```

对象的种类

BeautifulSoup 对象表示的是一个文档的全部内容. 大部分时候, 可以把它当作 Tag 对象, 它支持遍历文档树和搜索文档树中描述的大部分的方法.

因为 BeautifulSoup 对象并不是真正的 HTML 或 XML 的 tag, 所以它没有 name 和 attribute 属性. 但有时查看它的.name 属性是很方便的, 所以 BeautifulSoup 对象包含了一个值为 “[document]” 的特殊属性.name

```
soup.name  
# u'[document]'
```

对象的种类

Tag , NavigableString , BeautifulSoup 几乎覆盖了 html 和 xml 中的所有内容, 但是还有一些特殊对象. 容易让人担心的内容是文档的注释部分:

对象的种类

```
markup = "<b><!--Hey, buddy. Want to buy a used parser  
soup = BeautifulSoup(markup)  
comment = soup.b.string  
type(comment)  
# <class 'bs4.element.Comment'>
```

Outline

- 1 快速开始
- 2 对象的种类
- 3 遍历文档树
- 4 搜索文档树

遍历文档树

还拿" 爱丽丝梦游仙境" 的文档来做例子:

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>

<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and their names were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""

from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc)
```

遍历文档树

通过这段例子来演示怎样从文档的一段内容找到另一段内容

遍历方法

- 使用点号遍历，如 `doc.body.a` 代表 `body` 下第一个 `<a>` 连接
- `.descendants` 表示子节点
- `.next_element` 表示下一个节点
- `.next_sibling` 表示下一个兄弟
- `.name` 返回标签类型
- `.get_text()` 返回文本
- 复数表示生成器
- `find_all` 更好用
- `select` 也不错，支持 CSS 选择器

遍历文档树

```
soup.head
# <head><title>The Dormouse's story</title></head>

soup.title
# <title>The Dormouse's story</title>
```

遍历文档树

通过点取属性的方式只能获得当前名字的第一个 tag:

```
soup.a
```

```
# <a class="sister" href="http://example.com/elsie" id
```

遍历文档树

如果想要得到所有的 `<a>` 标签, 或是通过名字得到比一个 tag 更多的内容的时候, 就需要用到 Searching the tree 中描述的方法, 比如: `find_all()`

```
soup.find_all('a')  
# [<a class="sister" href="http://example.com/elsie" i  
#  <a class="sister" href="http://example.com/lacie" i  
#  <a class="sister" href="http://example.com/tillie"
```

遍历文档树

tag 的.contents 属性可以将 tag 的子节点以列表的方式输出:

```
head_tag = soup.head
head_tag
# <head><title>The Dormouse's story</title></head>

head_tag.contents
[<title>The Dormouse's story</title>]

title_tag = head_tag.contents[0]
title_tag
# <title>The Dormouse's story</title>
title_tag.contents
# [u'The Dormouse's story']
```


遍历文档树

BeautifulSoup 对象本身一定会包含子节点, 也就是说 `<html>` 标签也是 BeautifulSoup 对象的子节点:

```
len(soup.contents)
# 1
soup.contents[0].name
# u'html'
```

遍历文档树

通过 tag 的.children 生成器, 可以对 tag 的子节点进行循环:

```
for child in title_tag.children:  
    print(child)  
    # The Dormouse's story
```

遍历文档树

继续分析文档树, 每个 tag 或字符串都有父节点: 被包含在某个 tag 中通过.parent 属性来获取某个元素的父节点. 在例子“爱丽丝”的文档中,<head> 标签是 <title> 标签的父节点:

遍历文档树

```
title_tag = soup.title
title_tag
# <title>The Dormouse's story</title>
title_tag.parent
# <head><title>The Dormouse's story</title></head>
```

遍历文档树

文档 title 的字符串也有父节点:<title> 标签

```
title_tag.string.parent  
# <title>The Dormouse's story</title>
```

遍历文档树

文档的顶层节点比如 <html> 的父节点是 BeautifulSoup 对象:

```
html_tag = soup.html  
type(html_tag.parent)  
# <class 'bs4.BeautifulSoup'>
```

遍历文档树

BeautifulSoup 对象的.parent 是 None:

```
print(soup.parent)
# None
```

遍历文档树

通过元素的.parents 属性可以递归得到元素的所有父辈节点, 下面的例子使用了.parents 方法遍历了 <a> 标签到根节点的所有节点.

```
link = soup.a
link
# <a class="sister" href="http://example.com/elsie" id="sister" >Elsie
for parent in link.parents:
    if parent is None:
        print(parent)
    else:
        print(parent.name)

# p
# body
# html
# [document]
# None
```


遍历文档树

```
sibling_soup = BeautifulSoup("<a><b>text1</b><c>text2</c></a></body></html>")
print(sibling_soup.prettify())
```

```
# <html>
#   <body>
#     <a>
#       <b>
#         text1
#       </b>
#     <c>
#       text2
#     </c>
#   </a>
# </body>
# </html>
```

遍历文档树

因为 `<b>` 标签和 `<c>` 标签是同一层: 他们是同一个元素的子节点, 所以 `<b>` 和 `<c>` 可以被称为兄弟节点. 一段文档以标准格式输出时, 兄弟节点有相同的缩进级别. 在代码中也可以使用这种关系.

遍历文档树

```
last_a_tag = soup.find("a", id="link3")
last_a_tag
# <a class="sister" href="http://example.com/tillie" i

last_a_tag.next_sibling
# ' ; and they lived at the bottom of a well.'
```

Outline

- 1 快速开始
- 2 对象的种类
- 3 遍历文档树
- 4 搜索文档树**

搜索文档树

Beautiful Soup 定义了很多搜索方法, 这里着重介绍 2 个: `find()` 和 `find_all()`. 其它方法的参数和用法类似, 请读者举一反三.

使用 `find_all()` 类似的方法可以查找到想要查找的文档内容

介绍 `find_all()` 方法前, 先介绍一下过滤器的类型, 这些过滤器贯穿整个搜索的 API. 过滤器可以被用在 tag 的 `name` 中, 节点的属性中, 字符串中或他们的混合中.

最简单的过滤器是字符串. 在搜索方法中传入一个字符串参数, Beautiful Soup 会查找与字符串完整匹配的内容, 下面的例子用于查找文档中所有的 `<b>` 标签:

```
soup.find_all('b')  
# [<b>The Dormouse's story</b>]
```

搜索文档树

如果传入正则表达式作为参数,Beautiful Soup 会通过正则表达式来匹配内容. 下面例子中找出所有以 b 开头的标签, 这表示 <body> 和 标签都应该被找到:

```
import re
for tag in soup.find_all(re.compile("^b")):
    print(tag.name)
# body
# b
```

搜索文档树

下面代码找出所有名字中包含” t” 的标签:

```
for tag in soup.find_all(re.compile("t")):  
    print(tag.name)  
# html  
# title
```

搜索文档树

如果传入列表参数,Beautiful Soup 会将与列表中任一元素匹配的内容返回. 下面代码找到文档中所有 <a> 标签和 标签:

```
soup.find_all(["a", "b"])  
# [<b>The Dormouse's story</b>,  
#  <a class="sister" href="http://example.com/elsie" i  
#  <a class="sister" href="http://example.com/lacie" i  
#  <a class="sister" href="http://example.com/tillie"
```


搜索文档树

True 可以匹配任何值, 下面代码查找到所有的 tag, 但是不会返回字符串节点

```
for tag in soup.find_all(True):  
    print(tag.name)  
# html  
# head  
# title  
# body  
# p  
# b  
# p  
# a  
# a  
# a  
# p
```

搜索文档树

`find_all( name , attrs , recursive , text , **kwargs )`
`find_all()` 方法搜索当前 tag 的所有 tag 子节点, 并判断是否符合过滤器的条件. 这里有几个例子:

```
soup.find_all("title")
# [<title>The Dormouse's story</title>]

soup.find_all("p", "title")
# [<p class="title"><b>The Dormouse's story</b></p>]

soup.find_all("a")
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.find_all(id="link2")
# [<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>]

import re
soup.find(text=re.compile("sisters"))
# u'Once upon a time there were three little sisters; and their names were\n'
```

搜索文档树

name 参数可以查找所有名字为 name 的 tag, 字符串对象会被自动忽略掉.

```
soup.find_all("title")  
# [<title>The Dormouse's story</title>]
```

```
soup.find_all(id='link2')  
# [<a class="sister" href="http://example.com/lacie" i
```

搜索文档树

如果传入 href 参数,Beautiful Soup 会搜索每个 tag 的” href” 属性:

```
soup.find_all(href=re.compile("elsie"))  
# [<a class="sister" href="http://example.com/elsie" i
```

搜索文档树

使用多个指定名字的参数可以同时过滤 tag 的多个属性:

```
soup.find_all(href=re.compile("elsie"), id='link1')  
# [<a class="sister" href="http://example.com/elsie" id="link1">
```

搜索文档树

可以通过 `find_all()` 方法的 `attrs` 参数定义一个字典参数来搜索包含特殊属性的 tag:

```
data_soup.find_all(attrs={"data-foo": "value"})  
# [<div data-foo="value">foo!</div>]
```

搜索文档树

按照 CSS 类名搜索 tag 的功能非常实用, 但标识 CSS 类名的关键字 `class` 在 Python 中是保留字, 使用 `class` 做参数会导致语法错误. 从 Beautiful Soup 的 4.1.1 版本开始, 可以通过 `class_` 参数搜索有指定 CSS 类名的 tag:

```
soup.find_all("a", class_="sister")
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
```

搜索文档树

tag 的 class 属性是多值属性. 按照 CSS 类名搜索 tag 时, 可以分别搜索 tag 中的每个 CSS 类名:

```
css_soup = BeautifulSoup('<p class="body strikeout"></p>')
css_soup.find_all("p", class_="strikeout")
# [<p class="body strikeout"></p>]

css_soup.find_all("p", class_="body")
# [<p class="body strikeout"></p>]
```


搜索文档树

通过 text 参数可以搜搜文档中的字符串内容. 与 name 参数的可选值一样, text 参数接受字符串, 正则表达式, 列表, True . 看例子:

```
soup.find_all(text="Elsie")  
# [u'Elsie']
```

```
soup.find_all(text=["Tillie", "Elsie", "Lacie"])  
# [u'Elsie', u'Lacie', u'Tillie']
```

```
soup.find_all(text=re.compile("Dormouse"))  
[u"The Dormouse's story", u"The Dormouse's story"]
```

搜索文档树

虽然 `text` 参数用于搜索字符串, 还可以与其它参数混合使用来过滤 `tag.Beautiful Soup` 会找到 `.string` 方法与 `text` 参数值相符的 `tag`. 下面代码用来搜索内容里面包含 “Elsie” 的 `<a>` 标签:

```
soup.find_all("a", text="Elsie")  
# [
```

搜索文档树

`find_all()` 和 `find()` 只搜索当前节点的所有子节点, 孙子节点等.
`find_parents()` 和 `find_parent()` 用来搜索当前节点的父辈节点, 搜索方法与普通 tag 的搜索方法相同, 搜索文档搜索文档包含的内容. 我们从一个文档中的一个叶子节点开始:

搜索文档树

```
a_string = soup.find(text="Lacie")
a_string
# u'Lacie'

a_string.find_parents("a")
# [<a class="sister" href="http://example.com/lacie" i

a_string.find_parent("p")
# <p class="story">Once upon a time there were three l
# <a class="sister" href="http://example.com/elsie" i
# <a class="sister" href="http://example.com/lacie" i
# <a class="sister" href="http://example.com/tillie"
# and they lived at the bottom of a well.</p>
```



搜索文档树

Beautiful Soup 支持大部分的 CSS 选择器, 在 Tag 或 BeautifulSoup 对象的.select() 方法中传入字符串参数, 即可使用 CSS 选择器的语法找到 tag:

```
soup.select("title")  
# [<title>The Dormouse's story</title>]
```

```
soup.select("p nth-of-type(3)")  
# [<p class="story">...</p>]
```

搜索文档树

通过 tag 标签逐层查找:

```
soup.select("body a")  
# [#   <a class="sister" href="http://example.com/lacie"  
#   <a class="sister" href="http://example.com/tillie"  
  
soup.select("html head title")  
# [<title>The Dormouse's story</title>]
```

搜索文档树

通过 CSS 的类名查找:

```
soup.select(".sister")  
# [<a class="sister" href="http://example.com/elsie" i  
# <a class="sister" href="http://example.com/lacie" i  
# <a class="sister" href="http://example.com/tillie"
```

```
soup.select("[class~=sister]")  
# [<a class="sister" href="http://example.com/elsie" i  
# <a class="sister" href="http://example.com/lacie" i  
# <a class="sister" href="http://example.com/tillie"
```

搜索文档树

通过 tag 的 id 查找:

```
soup.select("#link1")  
# [<a class="sister" href="http://example.com/elsie" i  
  
soup.select("a#link2")  
# [<a class="sister" href="http://example.com/lacie" i
```


搜索文档树

通过是否存在某个属性来查找:

```
soup.select('a[href]')  
# [<a class="sister" href="http://example.com/elsie" i  
#   <a class="sister" href="http://example.com/lacie" i  
#   <a class="sister" href="http://example.com/tillie"
```

搜索文档树

通过属性的值来查找:

```
soup.select('a[href="http://example.com/elsie"]')
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>]

soup.select('a[href^="http://example.com/"]')
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.select('a[href$="tillie"]')
# [<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.select('a[href*=".com/el"]')
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>]
```